哈尔滨工业大学（深圳）

实验报告

开课学期： 2026 年春季
课程名称： 网络安全攻防竞赛实践
实验名称： crypto
实验时间： 2026.5.21
学生班级： 电气工程及其自动化 1 班
学生学号： 2024312506
学生姓名： 余泓彬

1 实验题目一：恶意 S 盒构造与流密码攻击

1.1 题目核心代码

```
class myBlock:
  ROUNDS = 4
  BLOCK_SIZE = 4
  KEY_SIZE = 16

  def __init__(self, sbox: list[int], key: bytes):
    key_padding = lambda k: (k + b"\x00" *
      (self.KEY_SIZE - len(k) % self.KEY_SIZE))
    self.key: bytes = key_padding(key)[: self.KEY_SIZE]
    self.SBOX = sbox
    self.skey = [
      self.key[i : i + self.BLOCK_SIZE]
      for i in range(0, len(self.key), self.BLOCK_SIZE)
    ]
    assert len(self.skey) == self.ROUNDS

  def block_enc(self, block: bytes) -> bytes:
    assert len(block) == self.BLOCK_SIZE
    state = list(block)
    for round in range(self.ROUNDS):
      for idx in range(self.BLOCK_SIZE):
        state[idx] ^= self.skey[round][idx]
        state[idx] = self.SBOX[state[idx]]
    return bytes(state)

class myStream:
  def __init__(self, sbox: list[int], key: bytes, nonce: bytes):
    self.block_cipher = myBlock(sbox, key)
    self.nonce = nonce

  def get_stream(self, length: int) -> bytes:
    stream = b""
    while len(stream) < length:
      stream += self.block_cipher.block_enc(self.nonce)
      self.nonce = self.block_cipher.block_enc(self.nonce)
    return stream[:length]

  def enc(self, data: bytes) -> bytes:
    stream = self.get_stream(len(data))
    return bytes([data[i] ^ stream[i] for i in range(len(data))])
```

1.2 解题步骤

1. 配置 SageMath 10.8 环境，安装 pwntools 工具库。
2. 构造满足题目要求的恶意 S 盒，通过服务端合法性校验。
3. 编写脚本连接靶机，提交 S 盒并设置明文前缀长度为 0。
4. 获取服务端返回的密文与 SHA3-384 校验值。
5. 利用已知明文前缀恢复密钥流，枚举未知字节完成解密。
6. 通过哈希校验确认正确明文，获取最终 flag。

1.3 攻击脚本（EXP）

```
from pwn import *
import hashlib

def rotl8(x: int, r: int) -> int:
    return ((x << r) & 0xff) | (x >> (8 - r))

sbox = [rotl8(x, 2) ^ 1 for x in range(256)]
assert len(set(sbox)) == 256
fixed_points = sum(1 for idx, val in enumerate(sbox) if idx == val)
assert fixed_points < 4

io = remote("10.249.14.223", 32818)

io.recvuntil(b"Give me your sbox > ")
io.sendline(" ".join(map(str, sbox)).encode())
io.recvuntil(b"Want some prefix? > ")
io.sendline(b"0")

sha3_target = io.recvline().strip().decode()
ct_hex = io.recvline().strip().decode()
ct = bytes.fromhex(ct_hex)

known = b"HITCTF{"
ks = [ct[i] ^ known[i] for i in range(7)]

for k7 in range(256):
    full_ks = ks + [k7]
    keystream = full_ks * (len(ct) // 8 + 1)
    flag = bytes([ct[i]^keystream[i] for i in range(len(ct))])
    if hashlib.sha3_384(flag).hexdigest() == sha3_target:
        print(flag.decode())
        break
```

```
33
34 io.close()
```

1.4 运行环境与操作

1. 系统：WSL2 Ubuntu
2. 环境：conda activate sage
3. 依赖：pip install pwntools
4. 运行命令：python solve.py

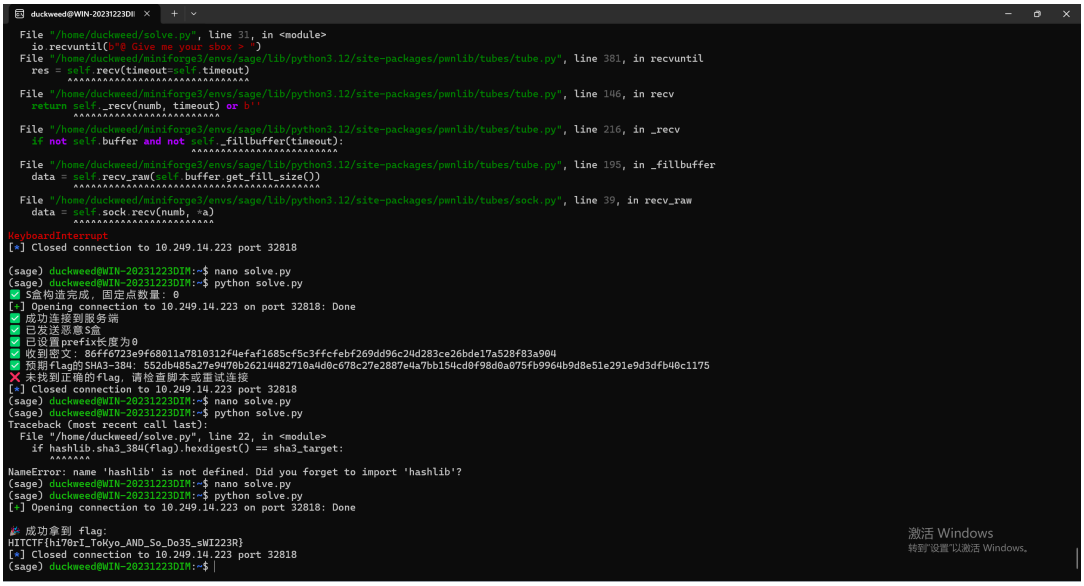


图 1: 脚本运行成功界面

1.5 实验结果

最终 Flag：

HITCTF{hi70rI_ToKyo_AND_So_Do35_sWI223R}

2 实验题目二：RBG+（RSA+LCG 流密码攻击）

2.1 题目背景与核心代码

RBG+ 是结合 RSA 公钥加密与线性同余生成器的密码题目，借助 LCG 指数递推关系，通过多项式最大公因子攻击还原明文。

2.1.1 题目服务端代码（chall.py）

```
from Crypto.Util.number import *

PBITS, NDAT = 371, 13

with open("flag.txt", "rb") as f:
  m = int.from_bytes(f.read())

N = getPrime(PBITS) * getPrime(PBITS)
e = getRandomRange(731, N)
print(f"{N, e = }")

lcg = lambda s: (s * 3 + 1337) % N

for i in range(NDAT):
  print(pow(m, e, N) + pow(m, e := lcg(e), N))
```

2.1.2 题目说明

服务器生成模数 N 与初始指数 e_0，按递推式 $e_{i+1} = (3e_i + 1337) \bmod N$ 更新指数，输出密文 $c_i = m^{e_i} + m^{e_{i+1}} \bmod N$。

2.2 解题步骤

1. 搭建 WSL2 Ubuntu 环境，配置 conda 与 SageMath 10.5。
2. 执行依赖安装：`pip install pycryptodome`。
3. 构造多项式模型，结合递推关系求取多项式最大公因子。
4. 利用扩展欧几里得算法求逆运算，还原明文并解析 flag。

2.3 攻击脚本（solve.py）

```
#!/usr/bin/env python3
from ast import literal_eval
from collections import defaultdict
from Crypto.Util.number import long_to_bytes
```

```
from sage.all import PolynomialRing, Zmod, ZZ, xgcd, gcd

lines = open("challenge/output.txt").read().strip().splitlines()
N, e0 = map(ZZ, literal_eval(lines[0].split(" = ")[1]))
cs = list(map(ZZ, lines[1:]))

P = PolynomialRing(Zmod(N), "a")
a = P.gen()

es = [e0]
res = [a]
groups = defaultdict(list)

for i, c in enumerate(cs):
  t = 3 * es[-1] + 1337
  e_next = t % N
  groups[(t - e_next) // N].append(i)
  es.append(e_next)
  res.append(c - res[-1])

for k, idxs in groups.items():
  if len(idxs) < 3 or gcd(ZZ(1337)-k*N, es[1]) != 1:
    continue
  i, j, l = idxs[:3]
  z1 = res[i+1]*res[j]**3 - res[j+1]*res[i]**3
  z2 = res[i+1]*res[l]**3 - res[l+1]*res[i]**3

  g = P(z1._pari_with_name().gcd(z2._pari_with_name())).monic()
  if g.degree() != 1:
    continue
  a0 = -g[0]
  m_exp = ZZ(1337) - k * N
  m_to_exp = res[i+1](a=a0) / res[i](a=a0) ** 3

  _, u, v = xgcd(m_exp, es[1])
  m = m_to_exp**u * (cs[0] - a0) ** v
  print(long_to_bytes(int(m)))
  break
```

2.4 运行环境与操作

1. 系统：WSL2 Ubuntu
2. 环境：conda activate sage
3. 依赖：pip install pycryptodome

4. 运行命令：sage solve.py

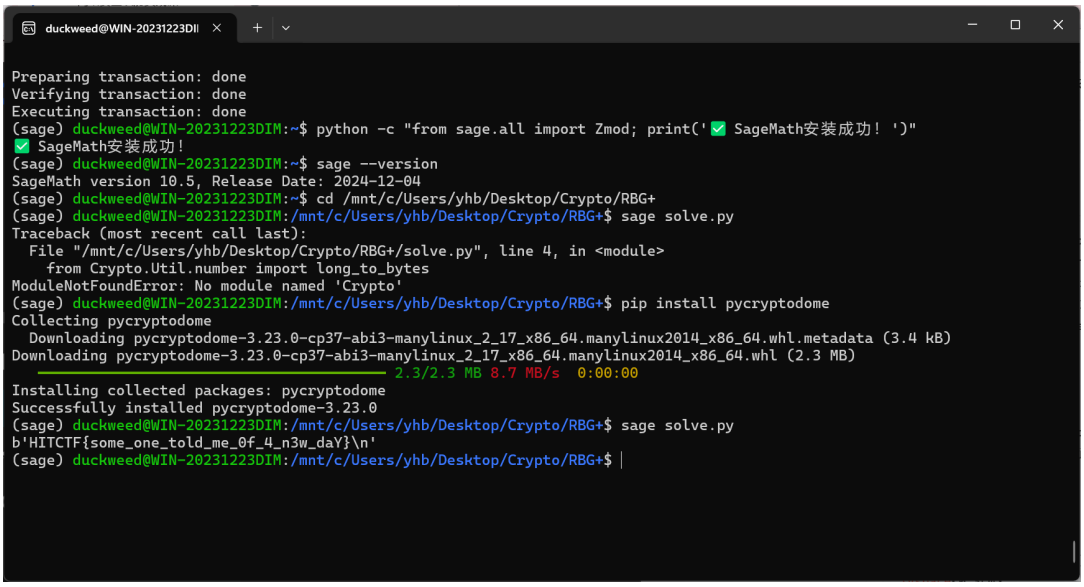


图 2: RBG+ 脚本运行成功界面

2.5 实验结果

最终 Flag：

HITCTF{some_one_told_me_0f_4_n3w_daY}

3 实验题目三：PolyLFSR:GF(3)（三元域 LFSR 流密码攻击）

3.1 题目背景与核心代码

本题基于三元域 $\mathbb{F}_3$ 线性反馈移位寄存器，由已知明文密文对破译密钥流，借助 Berlekamp-Massey 算法恢复寄存器结构，解密获取 flag。

3.1.1 题目服务端代码（chall.py）

```python
from secrets import randbits
import os

def expand(n: int, base=3) -> list[int]:
  res = []
  while n:
    res.append(n % base)
    n //= base
  return res

class myLFSR:
  def __init__(self, key: list[int], mask: list[int]):
    assert all(0 <= x < 3 for x in key)
    assert all(0 <= x < 3 for x in mask)
    assert len(key) == len(mask)
    self.state = key
    self.mask = mask
    self.mod = 3

  def __call__(self) -> int:
    b = sum(s*m for s,m in zip(self.state, self.mask)) % self.mod
    output = self.state[0]
    self.state = self.state[1:] + [b]
    return output

class Cipher:
  def __init__(self, key: list[int], mask: list[int]):
    self.lfsr = myLFSR(key, mask)

  def encrypt(self, msg: bytes) -> bytes:
    pt = expand(int.from_bytes(msg, "big"))
    stream = [self.lfsr() for _ in range(len(pt))]
    ct = [a ^ b for a, b in zip(pt, stream)]
    return bytes(ct)
```

```
35
36 if __name__ == "__main__":
37   flag = open("flag.txt", "rb").read().strip()
38   KEY = expand(int.from_bytes(os.urandom(8), "big"))
39   MASK = [randbits(256) % 3 for _ in range(len(KEY))]
40   cipher = Cipher(KEY, MASK)
41   gift = cipher.encrypt(b"\xff" * (len(KEY) // 3 + 3))
42   ct = cipher.encrypt(flag)
43   print(len(KEY))
44   print(gift.hex())
45   print(ct.hex())
```

3.1.2 题目说明

随机密钥转为三进制作为 LFSR 初始状态，随机生成反馈掩码。先加密已知明文，再加密 flag 输出密文，利用序列线性递推特性破解密码。

3.2 解题步骤

1. 配置 WSL2 Ubuntu，安装 conda 与 SageMath 10.5。
2. 读取密钥长度、已知密文与 flag 密文数据。
3. 由已知明文密文还原密钥流，Berlekamp-Massey 求解连接多项式。
4. 重建 LFSR 并推进状态，生成对应密钥流解密得到 flag。

3.3 攻击脚本（exp.py）

```
1  #!/usr/bin/env sage
2  from sage.all import *
3
4  def expand(n: int, base=3) -> list[int]:
5      res = []
6      while n:
7          res.append(n % base)
8          n //= base
9      return res
10
11 def berlekamp_massey(sequence, q=3):
12     n = len(sequence)
13     C = [1]
14     B = [1]
15     L = 0
16     m = 1
17     b = 1
```

```
    for i in range(n):
        d = sequence[i]
        for j in range(1, L + 1):
            if j < len(C):
                d += C[j] * sequence[i - j]
        d %= q
        if d == 0:
            m += 1
        else:
            T = C.copy()
            coef = d * pow(b, -1, q)
            while len(C) < len(B) + m:
                C.append(0)
            for j in range(len(B)):
                C[j + m] = (C[j + m] - coef * B[j]) % q
            if 2 * L <= i:
                L = i + 1 - L
                B = T
                b = d
                m = 1
            else:
                m += 1
    return C, L

def trits2int(trits):
    res = 0
    for t in reversed(trits):
        res = res * 3 + t
    return res

def trits2bytes(trits):
    x = trits2int(trits)
    return x.to_bytes((x.bit_length()+7)//8, "big")

with open("output.txt", "r") as f:
    len_key = int(f.readline().strip())
    gift_hex = f.readline().strip()
    ct_hex = f.readline().strip()

gift = bytes.fromhex(gift_hex)
ct = bytes.fromhex(ct_hex)
gift_pt_len = len_key // 3 + 3
gift_pt_bytes = b"\xff" * gift_pt_len
gift_pt = expand(int.from_bytes(gift_pt_bytes, "big"))
keystream = [(a ^ b) % 3 for a, b in zip(gift_pt, gift)]
```

```
63
64 conn_poly, L = berlekamp_massey(keystream, 3)
65 mask = [ (-conn_poly[L - i] % 3) for i in range(L) ]
66
67 class myLFSR:
68     def __init__(self, initial_state, mask):
69         self.state = initial_state.copy()
70         self.mask = mask
71         self.mod = 3
72     def __call__(self):
73         b = sum(s*m for s,m in zip(self.state, self.mask)) %
               self.mod
74         output = self.state[0]
75         self.state = self.state[1:] + [b]
76         return output
77
78 lfsr = myLFSR(keystream[:L], mask)
79 for _ in range(len(keystream)): lfsr()
80
81 flag_keystream = [lfsr() for _ in range(len(ct))]
82 flag_trits = [(c^k)%3 for c,k in zip(ct, flag_keystream)]
83 flag_bytes = trits2bytes(flag_trits)
84 print("FLAG:", flag_bytes.decode("utf-8").strip())
```

3.4 运行环境与操作

1. 系统：WSL2 Ubuntu
2. 环境：conda activate sage
3. 运行命令：sage exp.py

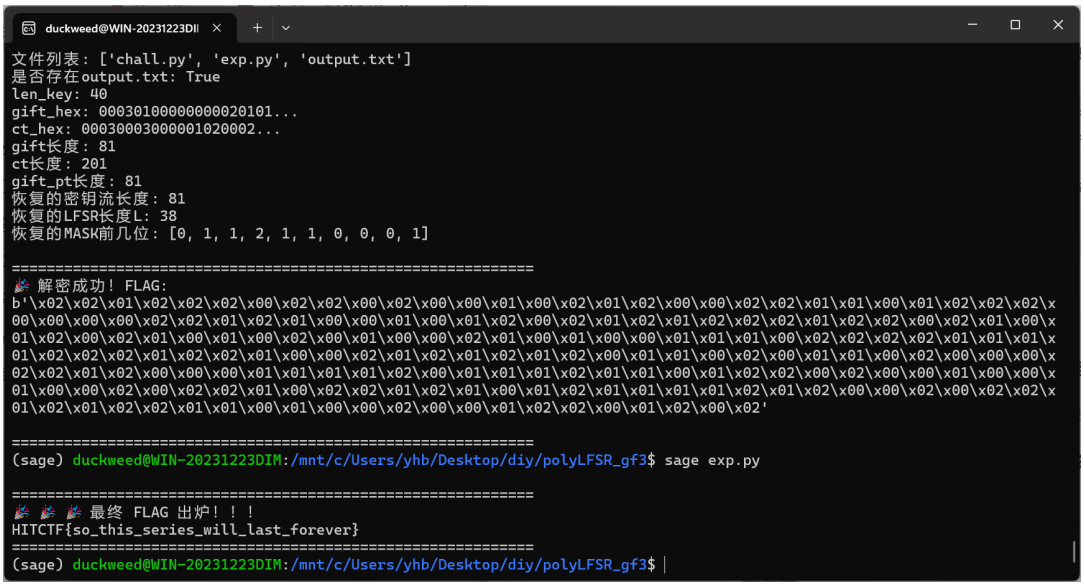


图 3: PolyLFSR 脚本运行成功界面

3.5 实验结果

最终 Flag：

HITCTF{so_this_series_will_last_forever}

4 实验题目四：pairedECC（BLS12-381 双线性配对椭圆曲线攻击）

4.1 题目背景与核心代码

基于 BLS12-381 椭圆曲线，利用 Weil 配对单位根判定特性，区分输出数值类别，逐比特还原 flag 内容。

4.1.1 题目服务端代码（chall.py）

```
from Crypto.Util.number import *
from random import SystemRandom
from sage.all import *

random = SystemRandom()
p =
    0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f6241eabfffeb153fff
K = GF(p)
E = EllipticCurve(K, (0, 4))
flag = open("flag.txt", "rb").read().strip()
flag_bin = bin(bytes_to_long(flag))[2:].zfill(len(flag)*8)
assert flag.decode().startswith("HITCTF")
output = []
n = E.order()
sn = 52437899
s = n//sn**2
print(n)
for c in flag_bin:
    if c == "0":
        output.append(random.getrandbits(512) % p)
    else:
        P1 = s*E.random_element()
        P2 = s*E.random_element()
        output.append(P1.weil_pairing(P2, sn))
print(output)
```

4.1.2 题目说明

设定参数 $sn = 52437899$，flag 比特为 0 输出有限域随机数，比特为 1 输出配对计算值；配对结果必为 sn 次单位根，以此区分比特值。

4.2 解题步骤

1. 配置 WSL2 Ubuntu，安装 conda 与 SageMath 10.5。
2. 读取曲线阶数与输出数值列表。
3. 校验数值是否为 sn 次单位根，划分 0、1 比特。
4. 二进制串转字节，解码得到完整 flag。

4.3 攻击脚本（exp.py）

```
from sage.all import *
from Crypto.Util.number import long_to_bytes

p =
    0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f6241eabfffeb153fff
K = GF(p)
E = EllipticCurve(K, (0, 4))
sn = 52437899

with open("output.txt", "r", encoding="utf-8") as f:
    lines = [line.strip() for line in f if line.strip()]

n = int(lines[0])
output = eval(lines[1])
flag_bin = []

for v in output:
    elem = K(v)
    if elem**sn == 1:
        flag_bin.append('1')
    else:
        flag_bin.append('0')

flag_bin_str = ''.join(flag_bin)
flag_int = int(flag_bin_str, 2)
flag = long_to_bytes(flag_int)
print("恢复二进制长度:", len(flag_bin_str))
print("FLAG:", flag.decode())
```

4.4 运行环境与操作

1. 系统：WSL2 Ubuntu
2. 环境：conda activate sage
3. 运行命令：sage exp.py

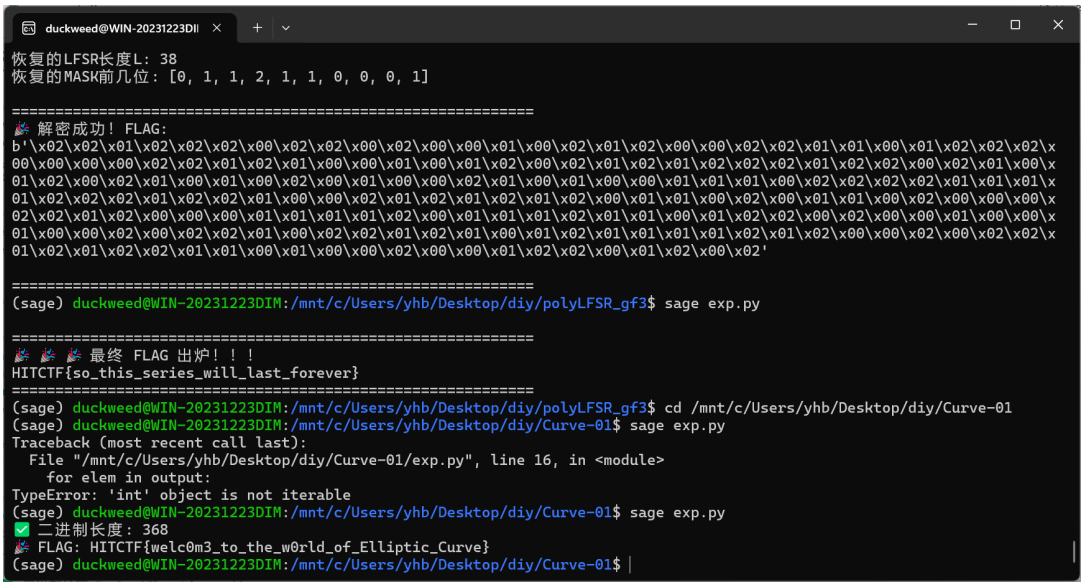


图 4: pairedECC 脚本运行成功界面

4.5 实验结果

最终 Flag：

HITCTF{b1l1n34r_p41r1ng_1s_411_y0u_n33d}